14-03-2026

Agenda :

Database - 2

surrogate keys → no real world meaning

row_id	student_id	name	email	phone	address	dept_id

artificial

dept_id	dept_name
1	—
2	—
3	—

student_id
student_id + name
student_id + email
email + phone
address row_id
⋮

super key : any combination (one or more) of columns that can uniquely identify a row. contain unnecessary data
→ SK → 50

candidate : Minimal super key

student_id →
student_id + email
student_id + row_id

CK → 25

primary : pick one to be the official identifier PK → 1/25

The alternate key : Candidate key − primary key
25 − 1
: 24

candidate key that wasn't picked as the primary key.

Foreign key → A column that refers to the primary key of another Table

: dept_id (Student) → dept_id (department)

Python + SQLlite

Supabase → api + UI → ≡ → cloud

postgre → DBMS
↓
DB

SQL-lite → DBMS → easy to use
↓
DB → python

SQL-lite :—

Steps:

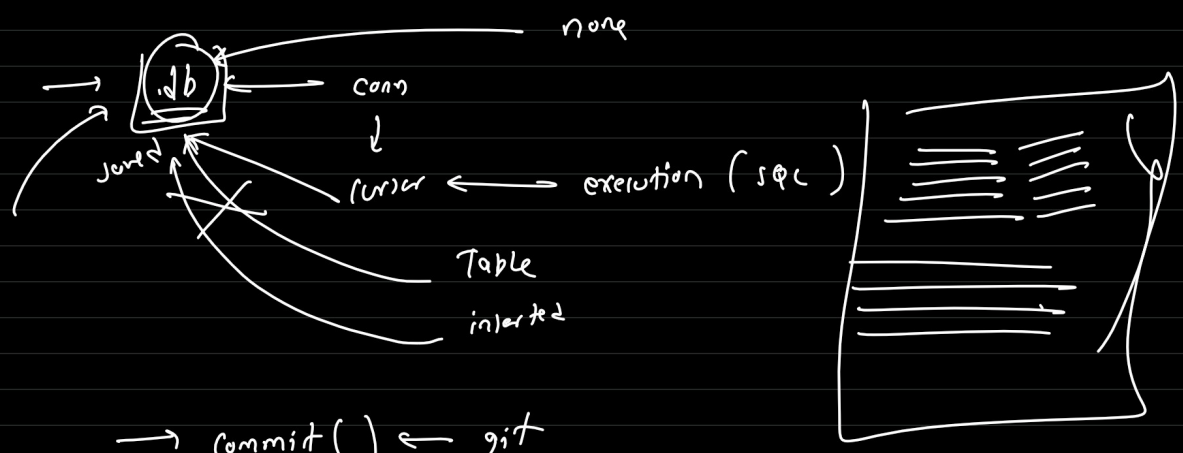
(1) Connect to database file (.db)
↳ if no file present
↳ create the database file
↓
conn ← object
↓
active connection
↑
cursor ← execute sql queries → 100
← fetch one
← fetch all

(2) connect connection with cursor object

cursor = conn.cursor()

cursor.execute(''' [redacted] ''')

(3)



→ commit() ← git

System design problem statement:

2 gb → ram

100 gb of storage

csv → 5 million record ← 5 x 10,00,000 → 8 gb

.db

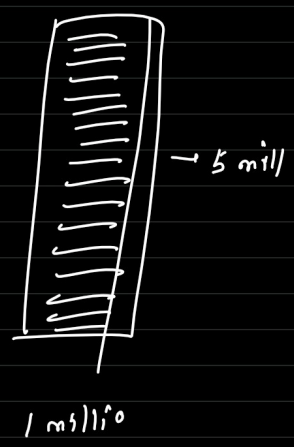
```
conn =

.csv        open( .csv , 'r' ) as f:
                for line in f:
                    conn.execute( insert )

for i in range( 0, 6 ):
    conn =
    curs =
```

→ 6

→ 6 times



```
open( .csv ,'r') as f:
    for index, line in enu(f):
        if index == 0:
            conn =
            curs =  → 6 time

        if indx % 100 == 0:
            conn.commit()
            conn =
            curs =  → 6 time
        else:
            curson.insert( line )

    conn.close()
```

Why can't we use PostgreSQL?

why we use sqlite when we have sql

sql → query language

sqlite → light weight DBMS

SELECT [ ] FROM ultimate_data_science ;

column name → all → *

Select * from ultimate_data_science ;

Supabase

→ postgres →

→ postgres → ← python →

→ sqlite →

→ mongodb → python

↑

Database → Schema → Table → colums + Data Types → Rows

Blueprint

↓

Primary keys

↓

Constraints ← Foreign keys

CRUD:

Create → Insert

Read → Select

Update → Update

Delete → Delete

SELECT (Read) : retrieve data from the table.

: not modify data

(1) select * from table_name;

* → all

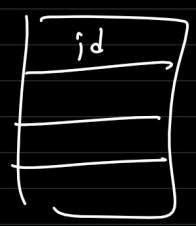
table_name → table

(2) select name, dept_id from employee;

Table: employee

id_employee_tb_1 emp_name_tb_1, sal_month

select id_employee_tb_1 emp_name_tb_1, sal_month from employees;

select id_employee_tb_1 as id, emp_name_tb_1 as name, sal_month as salary, from employees;

select id_employee_tb_1 as id from employees;